

- **Compiler** is software that translates source code written in a HLL into a set of machine language instructions (target program) that is understood by CPU.
- Compiler does not perform **multiplication** but **issue error messages**, know the **precedence** of operators and sometime optimize the source code.
- **Equivalent programs** means the output of those two programs are the same when they are given the same input.
- **Machine or assembly language** is difficult to work with and to maintain, machine dependent, does not support abstraction, encapsulation and polymorphism.
- **High-level language** the programmer does not have complete control of the machine's resources and compiler may generate inefficient machine language programs.
- **Interpreter** generates machine language program and carries out the computations.
- **Java language processors** combine compilation and interpretation.
- **A good compiler** has to generate fast and correct code, have linear compilation time, good diagnostics and work well with the debugger.

Using the big C notation, show each of the following compilers:

- (a) An Ada compiler which runs on the PC and compiles to the PC machine language.
- (b) An Ada compiler which compiles to the PC machine language, but which is written in Ada.
- (c) An Ada compiler which compiles to the PC machine language, but runs on a Sun.

### SOLUTION:

(a)

$$C_{PC}^{Ada \rightarrow PC}$$

(b)

$$C_{Ada}^{Ada \rightarrow PC}$$

(c)

$$C_{Sun}^{Ada \rightarrow PC}$$

- **big C notation**
- **Phases of a compiler** source program [lexical analysis] → tokens [syntax analysis] → atoms/syntax trees [code generation] → instructions [**local optimization**] → instruction.
- **Lexical analysis**/scanner attempts to isolate "words" or lexemes or lexical items or lexical tokens in an input string. It builds tables like symbol table that store **identifiers** used by subsequent phases. It does not check for proper syntax.
- **Types of words** key words, identifiers, operators, numeric constants, character constants, special characters like (|){ and comments.
- **Syntax analyzer** (parser) checks proper syntax, issue appropriate error messages and determine the underlying structure of the source program. Its output is stream of atoms that are atomic operations like MULT, ADD and MOVE or a collection of syntax trees.
- **Semantic analysis** (context handling) collects context information, checks for semantic errors like flow of controls, name uniqueness and incompatible operand and annotates nodes of the tree with the results.
- **Global optimization phase** used to make the object program more efficient in space or time by finding redundant or unnecessary instructions or inefficient code.

- In **code generation phase** atoms or syntax trees are translated to machine language (binary) instructions or to assembly language in which case the assembler is invoked to produce the object program. It's sometime responsible for register allocation.
- **NP-Complete problem:** limited number of register and instruction scheduling.
- **Local optimization** (machine-dependent) **phase** used to optimize object program that is put out by the code generator.

## Chapter two

- **Lexical analyzer** reads input characters of source program, group them into lexemes then produce a sequence of tokens for each lexeme. It strips out comments, whitespace & correlate error messages.
- **Formal language** is a set of strings that can be specified precisely and is amenable for use with computers.
- **A string** is a list of characters from a given **alphabet** (any finite set of symbols).
- **Finite state machine/automata theory** used to specify infinite language and consists of finite set of states and a state transition function.
- **Deterministic** each state there is exactly one arc leaving that state labeled by each possible input symbol. DFA is a 5-tuple  $M = (Q, \Sigma, \delta, q_0, F)$ .  $Q = q_1, q_2, \dots$ ,  $\Sigma = 0, 1$ ,  $\delta = Q * \Sigma$ ,  $q_0$  = initial &  $F$  = final.
- **Regular expressions** another method for specifying languages by involving **union (+)**, **concatenation**

$$\begin{aligned} L^0 &= \{\epsilon\} \\ L^1 &= L \\ L^2 &= L \cdot L \\ L^n &= L \cdot L^{n-1} \end{aligned}$$

(.) and kleene star or closure (\*).  $L^* = L^0 + L^1 + L^2 + L^3 + \dots$

$L(a^*) = \{\epsilon, a, aa, aaa, \dots\}$  (zero or more repetitions of a)

$L(aa^*) = \{a, aa, aaa, \dots\} = L(a^*a)$  (one or more repetitions of a)

$L((ab)^*) = \{\epsilon, ab, abab, ababab, \dots\}$  (zero or more repetitions of ab)

$L(a^*b^*) = \{\epsilon, a, aa, aaa, \dots, b, ab, aab, aaab, \dots, bb, abb, aabb, \dots\}$  (any number of a's followed by any number of b's)

$L((a+b)^*) = \{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, baa, bab, bba, \dots\}$  (zero or more repetitions of a's or b's, i.e., every string of a's and b's)

$L(a^* + b^*) = \{\epsilon, a, b, aa, bb, aaa, bbb, aaaa, bbbb, \dots\}$  (every string consisting entirely of a's or entirely of b's)

$L((a+b)(a+b)^*) = \{a, b, aa, ab, ba, bb, \dots\}$  (every nonempty string of a's and b's)

$L(a(ba+c)^*) = \{a, aba, ac, ababa, abac, acba, acc, \dots\}$

All strings of a's and b's that have the same first and last symbol:

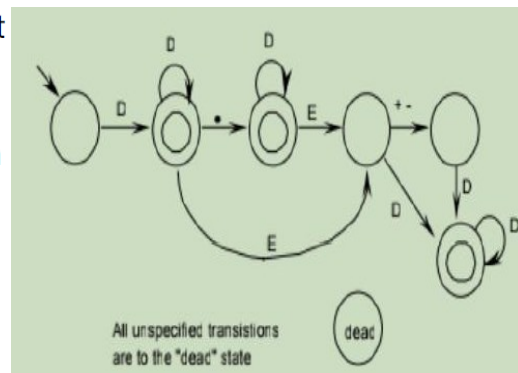
○  $\epsilon + a + b + a(a+b)^*a + b(a+b)^*b$

RE for  $L = \{\text{all strings of a's and b's that contain at least one a}\}$ :

○  $(a+b)^*a(a+b)^*$  or  $b^*a(a+b)^*$  or  $(a+b)^*ab^*$

RE for the set of all strings of zeros and ones which begin with a 1 and end with a 0

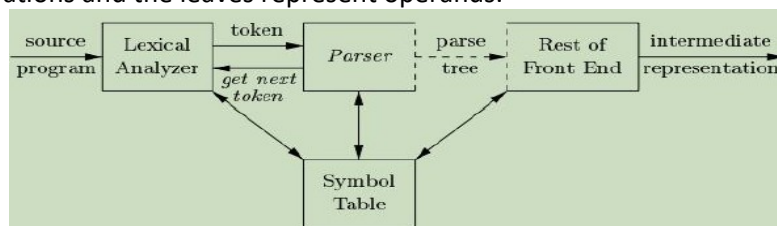
○  $1 \cdot (0+1)^* \cdot 0 = 1(0+1)^*0$



- During the lexical analysis constructed table stores identifier, what kind of identifier it is and value assigned to a variable once, regardless of the number of times it occurs in the source code.
- Type of data structure used is either binary search tree or hash table for efficiency in searching.
- Generated tokens consist a class indication which kind of token and a value indicating which member of the class.
- **Nondeterministic FSA** has no restrictions on the labels of their edges. Besides empty string is a possible label but can never be a member of input alphabet ( $\Sigma$ ).
- Every DFA is an NFA, but not vice versa i.e. there is an equivalent DFA for every NFA.
- **$\epsilon$ -closure**:  $\epsilon$ -closure for a given state A means a set of states which can be reached from the state A with only  $\epsilon$ (null) move including the state A itself.
- Executing instruction is up to the processor. **Front end process** (source program - global operation) and **backend process** (code generation – local operation).

### Chapter three **syntax analysis**

- **Syntax analysis** checks for proper syntax. It's not necessary for the compiler to generate an object program.
- **Atom** is a primitive operation and **syntax tree** is a data structure in which the interior nodes represent operations and the leaves represent operands.



- **Grammar** is a list of rules used to generate all the strings of a language, but does not generate any strings that are not in the language. It can used to specify the syntax of programming.L constructs.
- **Grammar** consists of finite set of characters (**input alphabet or input symbols or terminal symbols**), finite set of symbols (**nonterminal symbols**) and finite list of rewriting rules (**productions**).
- **Derivation** is a sequence of rewriting rules applied to the **starting nonterminal**, ending with a string of terminals. Used to demonstrate that a particular string is a member of the language.
- **Sentential form/ yield/ frontier of the tree** contains a mix of terminals and nonterminals.

|                                           |                                              |
|-------------------------------------------|----------------------------------------------|
| <b>A, B, C</b>                            | <b>A single nonterminal</b>                  |
| <b>a, b, c</b>                            | <b>A single terminal</b>                     |
| <b>X, Y, Z</b>                            | <b>A single terminal or nonterminal</b>      |
| <b>x, y, z</b>                            | <b>A string of terminals</b>                 |
| <b><math>\alpha, \beta, \gamma</math></b> | <b>A string of terminals and nonterminal</b> |

- There is no restriction in **unrestricted grammar** on the rewriting rules. Example:  $SaB \rightarrow cS$
- **Context sensitive** each rule must be of the form  $\alpha A \gamma \rightarrow \alpha \beta \gamma$ . Example:  $SaB \rightarrow caB$
- **Context free** each rule must be of the form  $A \rightarrow \alpha$ . Example:  $A \rightarrow aABb$ .
- **Right linear** each rule is of the form  $A \rightarrow aB$  or  $A \rightarrow a$ . used to define identifiers, constants and keywords.

|                                           |                          |
|-------------------------------------------|--------------------------|
| <b><math>aSb \rightarrow aAcBb</math></b> | <b>Context-Sensitive</b> |
| <b><math>B \rightarrow aA</math></b>      | <b>Right Linear</b>      |
| <b><math>a \rightarrow ABC</math></b>     | <b>Unrestricted</b>      |

|                |                          |
|----------------|--------------------------|
| <b>S → aBc</b> | <b>Context-Free</b>      |
| <b>Ab → b</b>  | <b>Context-Sensitive</b> |
| <b>AB → BA</b> | <b>Unrestricted</b>      |

- **BNF** representation for context free grammar:  $\langle S \rangle ::= a \langle S \rangle b \mid \epsilon$
- **Derivation or parse tree** is a tree in which each interior node corresponds to a nonterminal and each leaf node corresponds to a terminal symbol in the derived string.
- Context free is ambiguous when there is more than one derivation tree for a particular string.
- **Left-most or right-most derivation** solve the problem of ambiguity.

**A pushdown machine** is an NFA- $\epsilon$  with a stack. [pg. 43, 49]

- Regular languages/expression and finite state machines are for **lexical analysis**.
- Context free languages/grammar and pushdown automata are for **syntax analysis**.
- $M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$  where  $\delta: Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow$  finite subsets of  $Q \times \Gamma^*$ .
- In a **final state acceptability**, a PDA is in a final state after reading the entire string with any stack values.  $L(PDA) = \{w \mid (q_0, w, z_0) \vdash^* (q, \epsilon, x), q \in F\}$ .
- In **empty stack acceptability**, the PDA has emptied its stack after reading the entire string.  $L(PDA) = \{w \mid (q_0, w, I) \vdash^* (q, \epsilon, \epsilon), q \in Q\}$ .

Chapter four **syntax directed translation** [17, 23, 28, 40, 64, 67, 69, 94, and 100]

- In **translation grammar** action symbols is introduced that produce an output or call other methods while parsing an input string.
- **Rules** describe how the attributes are computed at specific node of the parse tree.
- **Syntax directed definition** specifies the value of attributes (data type/# of instructions/attached string) by associating semantic rules with each production i.e. grammar + semantic rules = SDT and a set of attributes with each grammar.
- **SDD applications** are infix into postfix, evaluate expression and build syntax tree.
- An **SDT scheme** is a notation for attaching program fragments/rules to the productions of a grammar that are executed as the production is used during syntax analysis.
- In **SDT scheme** the order of evaluation of the semantic rules is explicitly specified, this makes it different from SDD.
- **Semantic action** is program fragment embedded within production bodies. E.g. `{print('+')}`
- **SDD** associates grammar symbols with a set of **attributes** (t) and production with a set of **semantic rules** (||).
- **Synthesized attributes**: the idea of associating quantities with programming constructs.
- A parse tree showing the attribute values at each node is called an **annotated parse tree**.
- **Synthesized attribute's** value at a parse-tree node N is determined from attribute values at the children of N and at N itself.
- **Tree traversals** are used for describing attribute evaluation and for specifying the execution of code fragments in a translation scheme.
- **Parsing** determines whether an input string is in language of a given grammar.
- Parsing algorithms (**top down or bottom up**) refers to the sequence in which a derivation tree is built or traversed.
- Automating a language requires using **set and relations** (a set of ordered pairs).
- In **relation**: all pairs of 'R' are also in  $R^*$  and if (a, b) and (b, c) are in  $R^*$ ,  $\Rightarrow$  (a, c) is in  $R^*$  (Transitive).
- Context free **simple grammars**  $A \rightarrow a\alpha$  where a is any terminal and every pair of rules defining the same nonterminal has to begin with different terminals on the right side of the arrow..

- The set of input symbols that imply the application of a grammar rule is called the **selection set** of that rule.
- **Quasi-simple grammars**: obeys the restriction of simple grammars and also contain rules of the form  $A \rightarrow \epsilon$
- **LL (1) grammar** allows rule of context free grammar but two rules defining the same nonterminal must have disjoint selection sets.
  1. Find all nullable rules and nullable non terminals.
  2. Compute the relation "Begins Directly With" for each nonterminal.
  3. Compute the relation "Begins With".
  4. Compute the set of terminals "First(X)" for each symbol x in the grammar.
  5. Compute "First" of right side of each rule. If the grammar contains no nullable rules, you may skip to step 12 at this point.
  6. Compute the relation "Is Followed Directly By".
  7. Compute the relation "Is Direct End Of".
  8. Compute the relation "Is End Of".
  9. Compute the relation "Is Followed By". If  $W \rightarrow X, X \rightarrow Y, Y \rightarrow Z \rightarrow W \rightarrow Z$
  10. Extend the FB relation to include end marker.
  11. Compute the Follow Set for each nullable nonterminal.
  12. Compute the selection set for each rule.
- **Left recursion**  $A \rightarrow A\alpha$ : any grammar with left recursion cannot be LL (1).
- **Bottom-up parser** (shift reduce algorithm) an algorithms which proceed from the bottom of the derivation tree and apply grammar rules in a reverse order.
- If the input string is **acceptable**, the algorithm will terminate with staring terminal, alone on the stack and the grammar is called **LR**.
- the process of moving an input symbol to the stack is called a **shift** operation and the process of replacing symbols (**handle**) on the top of the stack with nonterminal is called a **reduce** operation.
- **Shift reduce conflict** occurs as the algorithm does not know whether to shift an input symbol or reduce the handle on the stack.
- **Reduce reduce conflict** occurs when it is clear that a reduce operation should be performed but it is not clear which rule should be used.
- **To resolve these conflicts**: making an assumption, rewrite the grammar and looking ahead at additional input characters help.

#### Chapter five **type checking and intermediate code generation** [19-20, 23-25]

- Source code  $\rightarrow$  front-end (parser $\rightarrow$ static checker $\rightarrow$ intermediate code generator)  $\rightarrow$  IR or intermediate code  $\rightarrow$ back-end (code generator).
- **Static checking** includes syntactic checking, L-values and R-values and type checking.
- Syntactic checking ensures whether a break statement is enclosed in loop/switch statement and identifier is declared at most once in a scope.
- Type checking ensures that operators are applied to compatible operands by assigning a type expression to each components of the source program.
- **Coercions** occurs if the type of an operand is automatically converted to the type expected by the operator. **Overloading operators** like '+'
- An intermediate representation may either be an actual language or it may consist of internal data structures that are shared by phases of the compiler.
- Syntax trees that are high level representations are suitable for type checking, while low level representation is suitable for register allocation and instruction selection.

- We use **IR** because it facilitates retargeting and enables machine independent code optimizations.
- Ease of manipulation, cost of manipulation, level of abstraction and ease generation are some of **design issues**.
- **Categories of IRs**: graphical/structural (**parse tree, abstract syntax tree and directed acyclic graph**), linear (3-address code and pseudo-code) and hybrid. These are code representation technics.
- Parse tree is not used b/c it yields a lot of unnecessary information. **To change it to AST**: traverse in post-order → use mkleaf and mknnode → match action with grammar rule.
- A node N in a **DAG** has more than one parent if N represents a common subexpression.
- **Analysis representations**: **control-flow graph, data dependence graph and call graph**.
- **CFG** models the way that the code transfers control b/n blocks in the procedure.
- **DDG** encodes the flow of data using **node** as a program statement and **edge** that connects two nodes if one uses the result of the other. It examine the legality of program transformations.
- **Call graph** shows dependences b/n procedures for inter-procedural analysis.
- **Three-address instruction** specifies the components of each type of instruction, but it does not specify the representation of these instructions in a data structure.